

Inference và triển khai (Inference and Deployment)

PHỤ THUỘC D/20-toi-uu-va-nen-mo-hinh (mô hình đã nén là đầu vào của chương này); B/11-gioi-han-tinh-toan (bảng thông, chỉ phí điều phối); B/08-dich-chuyen-phan-phoi (vì sao chỉ phân phối thật mới lộ ra lỗi thật); D/19/02-pytorch-thanh-thao (kỹ năng phân biệt “lệch do bug” với “lệch do số học”).

TÓM TẮT

Mô hình chạy tốt trong notebook và hỏng trong production. Chương này liệt kê từng chỗ nó hỏng, theo đúng trình tự bạn sẽ gặp: export, sai lệch số học sau export, thời gian nằm ở chỗ bạn không ngờ, xung đột giữa thông lượng và độ trễ đuôi, giới hạn nhiệt của thiết bị biên, và cuối cùng là rủi ro của việc thay mô hình. Đọc xong bạn thiết kế được một quy trình triển khai có kiểm chứng ở từng bước thay vì một chuỗi thao tác hý vọng. Nếu chỉ nhớ một điều: mô hình không có một con số độ trễ — *hệ thống ở một trạng thái* mới có, nên mọi phép đo phải nêu rõ trạng thái nguồn, nhiệt và tải kèm theo, và phải báo cáo p99 chứ không phải trung bình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **export** — quá trình chuyển mô hình từ mã Python/PyTorch sang một biểu diễn đồ thị tĩnh mà một *runtime* khác nạp được — ONNX là định dạng trung gian phổ biến nhất.
- **opset và plugin** — *opset* là phiên bản tập toán tử mà ONNX định nghĩa; toán tử của bạn không nằm trong đó thì phải viết *plugin* cho runtime, hoặc chấp nhận nó bị thay bằng một tổ hợp xấp xỉ.
- **engine** — kết quả biên dịch của TensorRT cho một tổ hợp cụ thể gồm phiên bản thư viện, kiến trúc GPU và cấu hình shape. Đồi bất kỳ thứ nào trong ba thứ đó là phải dựng lại.
- **dynamic shape và optimization profile** — shape đầu vào thay đổi lúc chạy (batch, độ phân giải). TensorRT đòi bạn khai báo trước dải shape (*optimization profile*) để biên dịch kernel phù hợp; không khai báo thì shape bị đóng băng lúc trace.
- **độ trễ p50 / p99** — phân vị của phân phối độ trễ — p99 là ngưỡng mà 99% yêu cầu nằm dưới. Trung bình che giấu đuôi, và đuôi mới là thứ người dùng cảm nhận.
- **batching động (dynamic batching)** — gom các yêu cầu đến gần nhau về thời gian thành một batch để tăng thông lượng, kèm một trần thời gian chờ để không phá ngân sách độ trễ.
- **KV cache** — bộ nhớ đệm lưu khoá và giá trị *attention* của các token đã sinh, để mỗi token mới không phải tính lại toàn bộ chuỗi. Đổi bộ nhớ lấy tốc độ.
- **shadow deployment** — chạy mô hình mới song song với mô hình cũ trên lưu lượng thật nhưng *không* dùng kết quả của nó để phục vụ; chỉ để so sánh và phát hiện lỗi trước khi chuyển đổi.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao con số độ trễ đo trong notebook gần như luôn sai khi lên thiết bị thật, và nó sai theo hướng nào?
- Q2. Bạn tối ưu lượt xuôi nhanh hơn 30% nhưng hệ thống chỉ nhanh hơn 7% — phần còn lại đã trốn ở đâu?
- Q3. Gộp *batch* làm thông lượng tăng gấp ba; vì sao nó vẫn có thể khiến hệ thống của bạn vi phạm ngân sách?

Q4. Cùng một engine, cùng một chip, cùng một đầu vào — vì sao phút thứ tám cho con số khác hẳn phút đầu tiên?

Q5. Sau khi export, đầu ra lệch 10^{-2} : làm sao biết đó là một bug hay chỉ là số học?

1.1 Đặt vấn đề

Chương này viết cho người đã triển khai thật, nên nó không giải thích lại độ trễ là gì. Câu hỏi dẫn dắt hẹp hơn và khó hơn: *mô hình chạy tốt trong notebook thì hỏng ở đâu khi ra production, và theo thứ tự nào?*

Điểm khiến việc triển khai deep learning khác với việc triển khai một thư viện C++ thông thường không phải độ phức tạp mà là *số lớp dịch*. Giữa mô hình bạn huấn luyện và thứ chạy trên thiết bị có ít nhất ba lần dịch — đồ thị Python sang ONNX, ONNX sang engine của runtime, engine sang kernel cụ thể — và mỗi lần dịch là một cơ hội để ngữ nghĩa thay đổi im lặng. Cộng thêm việc phần lớn phép đo hiệu năng trên GPU dễ đo sai theo những cách đã quen thuộc với người viết CUDA, và bạn có một hệ thống mà mọi con số đều phải được nghi ngờ cho tới khi có bằng chứng.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn đặt được một quy trình triển khai trong đó *mỗi bước có một phép kiểm chứng riêng*: so khớp từng tầng sau export, đo từng khối trong pipeline, đo p99 dưới tải nhiệt liên tục. Khi có sự cố, bạn khoanh vùng trong một giờ thay vì bởi cả chuỗi.

TRỰC GIÁC

Một mô hình không có độ trễ. Một hệ thống ở một trạng thái mới có độ trễ. Cùng một engine TensorRT trên cùng một Orin cho các con số khác nhau khi: chưa chạy lần nào (kernel autotune và cấp phát chưa xong), vừa chạy được ba mươi giây (mát, xung nhịp cao nhất), chạy liên tục tám phút (đã tụt xung vì nhiệt), hoặc chạy khi tiến trình SLAM bên cạnh đang ăn băng thông bộ nhớ chung.

Đây chính là bài học bạn đã học khi làm benchmark cho SLAM: một lần chạy không phải một phép đo, và hai lần chạy khác nhau không phải một sự bất thường mà là dữ liệu về phương sai. Sự khác biệt duy nhất là ở đây phương sai có thêm một nguồn vật lý — nhiệt độ — vốn có tính *trôi* chứ không phải *ngẫu nhiên*, nên nó không tự triệt tiêu khi bạn lấy trung bình nhiều lần lặp ngắn.

1.2 Mạch vấn đề → giải pháp, theo trình tự triển khai thật

Tám nút thắt, đúng thứ tự bạn sẽ gặp

1 Vấn đề

Giải pháp

Giá phải trả

PyTorch eager quá chậm và quá nặng để triển khai.

export sang một runtime chuyên dụng.

mất tính linh hoạt của Python và nhận về một tạo tác gắn chặt với phiên bản.

2 Vấn đề

Giải pháp

export xong thì đầu ra lệch.

so khớp từng tầng chứ không chỉ kết quả cuối, và kiểm tra bốn nguyên nhân theo đúng thứ tự.

- Giả định** bạn có một mẫu đầu vào cố định để so hai phía.
- 3 Vấn đề** **lượt xuôi** (*forward pass*) không phải chỗ tốn thời gian.
- Giải pháp** đo từng khối — giải mã, tiền xử lý, lượt xuôi, hậu xử lý — trước khi tối ưu bất cứ thứ gì.
- Hệ quả** trong hệ thống video, việc đưa giải mã và tiền xử lý lên GPU thường cho lợi ích lớn hơn mọi thứ bạn làm với mô hình.
- 4 Vấn đề** thông lượng và độ trễ xung đột.
- Giải pháp** batching động kèm trần thời gian chờ.
- Giá phải trả** p99 xấu đi khi hàng đợi dài, nên trần chờ là tham số thiết kế chứ không phải chi tiết cài đặt.
- 5 Vấn đề** mô hình sinh và VLM có cấu trúc chi phí khác hẳn.
- Giải pháp** KV cache, giảm số bước sampler, chưng cất sampler, giải mã suy đoán.
- Hệ quả** lại là mẫu hình “đẩy chi phí sang training-time” của chương 17 và 20.
- 6 Vấn đề** vận hành.
- Giải pháp** health check, autoscaling, warm-up bắt buộc, và báo cáo cả p50 lẫn p99.
- Giả định** bạn chấp nhận rằng lần chạy đầu luôn chậm và không cố giấu nó bằng cách bỏ qua mẫu đầu.
- 7 Vấn đề** thiết bị biên có giới hạn năng lượng và nhiệt.
- Giải pháp** đo trong mười phút liên tục ở đúng chế độ nguồn sẽ triển khai, không đo một trăm lần lặp.
- Hệ quả** benchmark ngắn nói dối một cách có hệ thống, luôn theo hướng lạc quan.
- 8 Vấn đề** thay mô hình mới là một canh bạc.
- Giải pháp** shadow deployment rồi A/B test.
- Giả định** chỉ phân phối thật mới lộ ra các lỗi mà tập test không có (B/08-dịch-chuyen-phan-phoi).

1.3 Export và chọn runtime

A. Bón họ runtime

Bảng 1.1. Bón họ runtime và cái mà mỗi cái đánh đổi.

Runtime	Tối ưu cho	Đánh đổi	Hỏng khi nào
ONNX Runtime	Trung lập, chạy được nhiều nơi	Không nhanh nhất ở bất kỳ đâu	Khi bạn thực sự cần ép sát ngân sách trên một chip cụ thể
TensorRT	GPU và DLA của NVIDIA	Khoá nhà cung cấp; build lâu; engine gắn với phiên bản và kiến trúc	Toán tử lạ phải viết plugin; nâng cấp JetPack là phải dựng lại toàn bộ
OpenVINO	CPU và iGPU của Intel	Hẹp về phần cứng	Ngoài hệ sinh thái Intel
ExecuTorch / CoreML / TFLite	Di động và nhúng	Tập toán tử hạn chế hơn hẳn	Mô hình dùng op động hoặc điều khiển luồng phức tạp

Với một hệ thống thời gian thực trên Orin, hai chi tiết quyết định nhiều hơn bảng trên. Thứ nhất, engine TensorRT phải được *đựng trên chính thiết bị đích* hoặc trên một môi trường trùng khít về phiên bản, vì nó nhúng cả lựa chọn kernel lẫn giả định về kiến trúc GPU. Thứ hai, Orin có DLA bên cạnh GPU; đẩy một phần đồ thị xuống DLA giải phóng GPU cho phần khác của hệ thống, nhưng DLA hỗ trợ ít toán tử hơn nên phần không chạy được sẽ rơi ngược về GPU và tạo ra các lần chuyển đổi tốn kém giữa hai bộ máy.

B. Sai lệch số học sau export — bốn nguyên nhân theo thứ tự

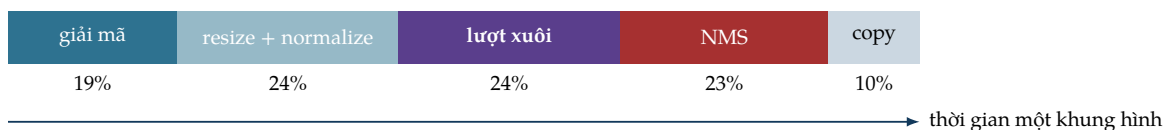
Đây là bước mà kỹ thuật quan trọng hơn kỹ thuật: *so khớp đầu ra từng tầng sau mỗi lần export, không chỉ so kết quả cuối*. Lý do giống hết lý do kiểm tra gradient từng tầng ở chương 19 — giữ không gian nghi vấn ở một phần tử. Khi đã thấy chỗ lệch, kiểm theo thứ tự này:

↪ **cùng nguyên lý với D/19/01** — so khớp từng tầng sau export là đúng một lập luận với kiểm tra gradient từng tầng: giữ không gian nghi vấn ở một phần tử thay vì ở tích các tương tác.

1. **Dynamic shape bị cố định lúc trace.** Triệu chứng đặc trưng: đúng với đúng một kích thước đầu vào, sai hoặc lỗi với mọi kích thước khác. Đây là nguyên nhân phổ biến nhất và cũng rẻ nhất để loại trừ.
2. **Toán tử không được hỗ trợ bị thay bằng bản xấp xỉ.** Triệu chứng: lệch tập trung ở một tầng cụ thể và không giảm khi bạn tăng độ chính xác. Kiểm bằng cách đọc log export — nó thường có cảnh báo mà không ai đọc.
3. **Khác biệt thứ tự phép toán ở fp16.** Triệu chứng: lệch nhỏ, phân tán đều, cỡ epsilon của fp16 và tích lũy dần theo độ sâu. Đây là *số học*, không phải bug — và phân biệt được hai thứ này là kỹ năng bạn đã luyện ở mini project của chương 19.
4. **Tiền xử lý khác nhau giữa hai phía.** Thứ tự kênh BGR so với RGB, giá trị mean/std, cách resize (song tuyến tính có căn góc hay không), và việc chia cho 255 hay không. Triệu chứng: lệch lớn và có hệ thống ngay từ tầng đầu tiên. Nghe tầm thường, nhưng đây là nguyên nhân tốn nhiều giờ người nhất trong danh sách, vì nó nằm ngoài mô hình nên không ai nghĩ tới nó khi đang debug mô hình [1].

1.4 Chỗ thời gian thực sự nằm

Kỳ vọng mặc định là lượt xuôi chiếm phần lớn thời gian. Trong một hệ thống thị giác thật, kỳ vọng đó thường sai. Hình 1.1 minh họa một phân rã điển hình cho một bộ phát hiện chạy trên luồng video — các con số là minh họa cho hình dạng, không phải một phép đo cụ thể.



Hình 1.1. Phân rã thời gian điển hình của một bộ phát hiện trên luồng video (*hình dạng minh họa, không phải số đo*). Phần được viết paper — lượt xuôi — thường chiếm khoảng một phần tư. Tối ưu 30% của một phần tư cho lợi ích tổng 7%; đưa giải mã và tiền xử lý lên GPU cho lợi ích lớn hơn nhiều, và đó là lý do các bộ công cụ như DALI hay DeepStream tồn tại.

Hai chỗ đáng chú ý riêng. **Triệt phi cực đại (non-maximum suppression, NMS)** là hậu xử lý tuần tự và phụ thuộc dữ liệu. Chi phí của nó tăng theo số hộp *trước* ngưỡng. Nên một ngưỡng tin cậy đặt thấp làm p99 xấu đi nhiều hơn p50, đúng ở những khung hình đông đúc mà bạn quan tâm nhất [2]. **Sao chép giữa CPU và GPU** thường bị bỏ qua hoàn toàn trong phép đo; trên Orin, bộ

nhớ hợp nhất cho phép tránh phần lớn các bản sao này, nhưng chỉ khi bạn cấp phát đúng loại bộ nhớ ngay từ đầu — một quyết định phải làm ở lúc thiết kế chứ không sửa được về sau.

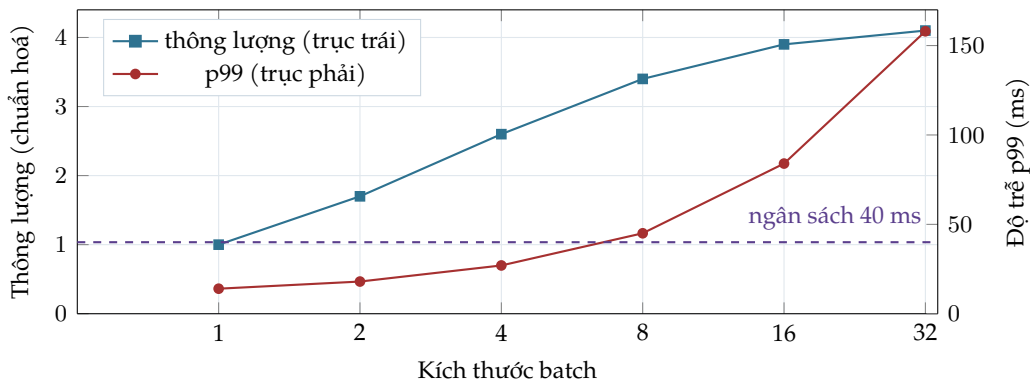
↪ **ràng buộc bởi B/11** — trên Orin, GPU và CPU chia chung băng thông bộ nhớ, nên một tiến trình khác đang chạy đủ để đổi con số độ trễ mà không đổi một dòng mã nào.

1.5 Batching động và ngân sách độ trễ

Gộp batch làm thông lượng tăng và độ trễ tăng. Cách đúng để nghĩ về nó là coi trần thời gian chờ là một tham số thiết kế, rồi tính xem tốc độ đến của yêu cầu có đủ để lấp batch trong trần đó không.

Ví dụ nhỏ nhất tính tay được. Ngân sách p99 là 40 ms. Forward cho batch 8 tốn 20 ms, nên còn 20 ms cho hàng đợi. Muốn lấp đầy batch 8 trong 20 ms thì tốc độ đến phải ít nhất $8/0,02 = 400$ yêu cầu mỗi giây. Nếu thực tế chỉ có 100 yêu cầu mỗi giây, trong 20 ms bạn chỉ gom được 2 yêu cầu — nên bạn đang trả cái giá về độ trễ của batching mà chỉ nhận được lợi ích của batch 2.

Kết luận thiết kế: batching động chỉ có lãi khi tốc độ đến vượt ngưỡng $B/T_{\text{chờ}}$. Dưới ngưỡng đó, cấu hình đúng là batch nhỏ hoặc không batch, và mọi nỗ lực chỉnh tham số batching là vô ích. Hình 1.2 cho thấy hình dạng của cuộc đánh đổi.



Hình 1.2. Thông lượng bão hoà trong khi p99 tiếp tục tăng gần tuyến tính theo kích thước batch (số liệu minh hoạ). Sau batch 8 bạn gần như không mua thêm được thông lượng nhưng vẫn trả đủ giá về độ trễ — và đúng ở đó ngân sách bị phá.

1.6 Mô hình sinh và VLM — cấu trúc chi phí khác hẳn

Với mô hình sinh tự hồi quy, chi phí không nằm ở một lượt xuôi mà ở n lượt, mỗi lượt sinh một *token*. KV cache biến bài toán từ “tính lại toàn bộ chuỗi mỗi bước” thành “chỉ tính token mới”: đổi bộ nhớ lấy tốc độ. Nhưng lượng bộ nhớ ấy tăng tuyến tính theo độ dài chuỗi và theo số yêu cầu đồng thời. Nó vì thế trở thành ràng buộc chính của việc phục vụ. Với mô hình khuếch tán, đòn bẫy tương ứng là giảm số bước sampler và chứng cất sampler; với LLM là giải mã suy đoán. Cả ba đều là cùng một mẫu hình: chuyển công việc từ lúc chạy sang lúc huấn luyện.

Điểm cần nhớ cho một hệ thống robotics: đưa một VLM vào vòng điều khiển thời gian thực là một quyết định kiến trúc, không phải một tối ưu. Độ trễ của nó phụ thuộc độ dài đầu ra, tức phụ thuộc *nội dung*, nên nó không có chặn trên tự nhiên — và mọi thành phần thời gian thực nổi vào nó phải được thiết kế với một cơ chế hết giờ và một hành vi dự phòng.

1.7 Vận hành và cách đo cho đúng

Ba nguyên tắc, nêu ngắn vì bạn đã biết chúng ở phía C++:

- **Warm-up là bắt buộc, không phải mẹo.** Lần chạy đầu gánh cấp phát bộ nhớ, autotune kernel của cuDNN, và nạp trọng số vào cache. Bỏ vài chục lần lặp đầu là chuẩn mực; không bỏ thì bạn đang báo cáo một phân phối trộn hai chế độ.
- **Đo phân vị, không đo trung bình.** Trung bình che giấu đúng thứ làm người dùng khó chịu và làm hệ thống điều khiển mất ổn định. p99 là con số ràng buộc thiết kế; p50 chỉ để biết hệ thống có đang lành mạnh không.
- **Đồng bộ hoá trước khi bấm giờ.** Lệnh CUDA là bất đồng bộ; một phép đo bằng đồng hồ tường quanh lời gọi chỉ đo thời gian xếp hàng.

1.8 Thiết bị biên: nơi benchmark ngắn nói dối

Trên Jetson và các NPU di động, hai ràng buộc vật lý chi phối mọi thứ: ngân sách năng lượng và tản nhiệt. Hệ quả trực tiếp là *một trăm lần lặp không phải một phép đo*. Chip chạy ở xung nhịp cao khi mát, rồi tụt xung khi nhiệt độ chạm ngưỡng; nếu vòng lặp benchmark của bạn kết thúc trước lúc đó, bạn đo được một con số mà hệ thống sẽ không bao giờ duy trì được.

Quy trình đo tối thiểu cho một thiết bị biên gồm bốn điều kiện, và cả bốn đều phải ghi vào báo cáo: chạy *liên tục ít nhất mười phút*; ở *đúng chế độ nguồn* sẽ triển khai (trên Orin là `nvpmodel`, và cần biết rằng `jetson_clocks` không tự giữ qua lần khởi động lại); *cùng với các tiến trình khác* sẽ chạy song song, vì chúng chia sẻ băng thông bộ nhớ; và ghi lại nhiệt độ cùng xung nhịp theo thời gian chứ không chỉ ghi độ trễ. Nếu đường độ trễ theo thời gian có một bậc thang, bạn vừa tìm thấy ngưỡng nhiệt — và con số đúng để báo cáo là con số *sau* bậc thang đó.

1.9 Thay mô hình an toàn

Thay mô hình mới là canh bạc vì tập test của bạn, dù tốt tới đâu, cũng không phải phân phối thật (B/08-dịch-chuyen-phan-phoi). Quy trình giảm rủi ro có hai bước và thứ tự không đảo được: **shadow deployment** — chạy song song trên lưu lượng thật, ghi lại kết quả nhưng không dùng chúng — rồi **A/B test** trên một phần lưu lượng. Shadow bắt được các lỗi kỹ thuật (tiền xử lý sai, shape sai, độ trễ vượt ngân sách, phân phối đầu ra lệch) mà không gây thiệt hại; A/B mới đo được tác động thật lên chỉ số nghiệp vụ. Bỏ qua bước shadow để đi thẳng vào A/B là cách phổ biến để phát hiện một lỗi tiền xử lý bằng chính người dùng.

↔ **tiền đề** B/08 — tập test là một mẫu cố định của quá khứ; chỉ lưu lượng thật mới bộc lộ các vùng phân phối đã trôi kể từ lúc thu dữ liệu.

1.10 Thực hành

Repo và tài nguyên. *Runtime:* [microsoft/onnxruntime](#), [NVIDIA/TensorRT](#), [openvinotoolkit/openvino](#), [pytorch/executorch](#). *Phục vụ:* [triton-inference-server/server](#), [Lightning-AI/LitServe](#). *Pipeline video:* [NVIDIA/DALI](#) và [DeepStream](#). *Phục vụ mô hình sinh*

và VLM: [vllm-project/vllm](https://github.com/vllm-project/vllm), [sgl-project/sglang](https://github.com/sgl-project/sglang). Danh sách chốt tại tháng 9/2026; các runtime hạ tầng ở đây bền hơn nhiều so với tên các bộ phục vụ LLM, vốn thay đổi từng quý.

Câu hỏi kiểm tra hiểu. Sau khi export ONNX, đầu ra lệch 10^{-2} so với PyTorch — bốn nguyên nhân theo thứ tự cần kiểm là gì, và triệu chứng phân biệt chúng? Vì sao p99 mới là con số ràng buộc thiết kế chứ không phải trung bình? Batching động cải thiện thông lượng nhưng làm hỏng gì, và ngưỡng tốc độ đến tối thiểu để nó có lãi được tính thế nào?

Mini project (vài giờ đến hai ngày). Export một bộ phát hiện sang ONNX rồi sang TensorRT. Ở mỗi bước, kiểm tra khớp đầu ra *từng tầng* và đo p50 cùng p99. Lập bảng thời gian theo từng khối: giải mã, tiền xử lý, lượt xuôi, NMS, sao chép. Kết quả đáng giá nhất của bài này thường là phát hiện rằng khối chiếm nhiều thời gian nhất không phải khối bạn định tối ưu.

1.11 Giới hạn và trường hợp hỏng

- **Engine gắn chặt với phiên bản.** Nâng JetPack, đổi phiên bản TensorRT, hay đổi biến thể GPU là phải dựng lại và *kiểm chứng lại* — không phải chỉ dựng lại. Đây là chi phí vận hành thường xuyên chứ không phải chi phí một lần.
- **So khớp từng tầng không bắt được lỗi phụ thuộc dữ liệu.** Một mẫu đầu vào cố định không kích hoạt nhánh điều khiển hiếm; NMS với số hộp bất thường, hoặc một shape ngoài optimization profile, vẫn có thể hỏng sau khi mọi phép so đã qua.
- **p99 đo trên lưu lượng tổng hợp không dự báo p99 thật.** Lưu lượng thật có cụm và có tương quan; một bộ sinh yêu cầu đều đặn cho phân phối hàng đợi lạc quan hơn nhiều so với thực tế.
- **Shadow deployment không phát hiện được vấn đề về vòng phản hồi.** Nếu mô hình mới thay đổi hành vi hệ thống và hành vi đó thay đổi phân phối đầu vào, chỉ A/B mới thấy — shadow theo định nghĩa không tác động gì nên không tạo ra vòng lặp đó.

CẠM BÃY

Ba cách đo sai làm bạn tin vào một con số không tồn tại. **Một:** bấm giờ bằng đồng hồ tường quanh một lời gọi CUDA mà không đồng bộ hoá — bạn đo thời gian *xếp hàng lệnh*, thường nhỏ hơn thời gian chạy thật một bậc. **Hai:** không warm-up, rồi lấy trung bình gồm cả lần chạy đầu vốn gánh autotune và cấp phát; con số ra vừa sai vừa có phương sai lớn giả tạo. **Ba:** benchmark một trăm lần lặp trên thiết bị biên đang mát, rồi báo cáo con số đó như hiệu năng bền vững. Hệ thống sẽ chạy chậm hơn ngay khi vào ca làm việc thật. Và không ai tái hiện được vấn đề, vì mọi người đều benchmark theo cách cũ.

CẦN NHỚ

Một phép đo độ trễ chỉ có nghĩa khi kèm trạng thái: chế độ nguồn, nhiệt độ đã bão hoà chưa, có tiến trình nào chạy cùng không, đã warm-up chưa, và đó là phân vị nào. Thiếu bất kỳ mục nào trong danh sách đó thì con số không tái hiện được và không dùng để ra quyết định được. Và nguyên tắc bao trùm cả chương: sau mỗi lần dịch — Python sang ONNX, ONNX sang engine — phải có một phép kiểm chứng riêng, vì mỗi lần dịch là một cơ hội để ngữ nghĩa đổi im lặng. [T2]

■ **Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Export	Chuyển mô hình từ mã Python sang một đồ thị tĩnh mà runtime khác nạp được; ONNX là định dạng trung gian phổ biến nhất. Mỗi lần dịch là một cơ hội đổi ngữ nghĩa im lặng.
●● Opset và plugin	Opset là phiên bản tập toán tử ONNX; plugin là phần bạn phải tự viết khi runtime không có op đó. Không viết thì op bị thay bằng một xấp xỉ.
●●● Engine TensorRT	Kết quả biên dịch cho một tổ hợp cụ thể gồm phiên bản thư viện, kiến trúc GPU và dải shape. Đối bất kỳ thứ nào là phải dựng lại và kiểm chứng lại.
●● Optimization profile	Khai báo trước dải shape động để TensorRT sinh kernel phù hợp. Không khai báo thì shape bị đóng băng lúc trace.
●● Vì sao phải so khớp đầu ra từng tầng sau export?	Để giữ không gian nghi vấn ở một tầng thay vì cả chuỗi ba lần dịch. Cùng lập luận với kiểm tra gradient từng tầng ở chương 19.
●●● Lệnh sau export	Theo thứ tự kiểm: shape động bị cố định lúc trace; op không hỗ trợ bị thay bằng xấp xỉ; khác thứ tự phép toán ở fp16; và tiền xử lý lệch giữa hai phía.
●●● Triệt phi cực đại	Hậu xử lý tuần tự, phụ thuộc dữ liệu, chi phí tăng theo số hộp trước ngưỡng. Ngưỡng tin cậy thấp làm p99 xấu đi nhiều hơn p50.
●●● Vì sao phải warm-up trước khi đo?	Lần chạy đầu gánh cấp phát bộ nhớ, autotune kernel và nạp trọng số vào cache. Không bỏ chúng đi thì bạn báo cáo một phân phối trộn hai chế độ.
●●● Vì sao p99 mới là con số ràng buộc thiết kế?	Vì trung bình che đậy cái đuôi mà người dùng cảm nhận và mà hệ điều khiển phải chịu. p50 chỉ để biết hệ thống có đang lành mạnh không.
●● Batching động	Gom yêu cầu đến gần nhau thành một batch, kèm trần thời gian chờ. Chỉ có lỗi khi tốc độ đến vượt $B/T_{chờ}$.
●● KV cache	Lưu khoá và giá trị attention của các token đã sinh để không tính lại chuỗi mỗi bước. Bộ nhớ tăng tuyến tính theo độ dài chuỗi và số yêu cầu đồng thời.
●● DLA trên Orin	Bộ máy suy luận riêng bên cạnh GPU, dùng để giải phóng GPU cho phần khác của hệ thống. Hỗ trợ ít op hơn, nên phần không chạy được rơi ngược về GPU.
●● Vì sao bấm giờ quanh một lời gọi CUDA cho số sai?	Lệnh CUDA bất đồng bộ, nên đồng hồ tường chỉ đo thời gian xếp hàng. Phải đồng bộ hoá hoặc dùng CUDA event.
● Shadow deployment	Chạy mô hình mới song song trên lưu lượng thật nhưng không dùng kết quả. Bắt lỗi kỹ thuật mà không gây thiệt hại; không bắt được vấn đề vòng phản hồi.
●● Thông lượng tăng gấp ba mà p99 vượt ngân sách?	Batch quá lớn. Thông lượng bão hoà trong khi p99 vẫn tăng gần tuyến tính, nên sau một điểm bạn trả giá mà không mua thêm được gì.
●● Đầu ra đúng với một kích thước ảnh, sai với mọi kích thước khác?	Shape động đã bị cố định lúc trace, hoặc thiếu optimization profile. Đây là nguyên nhân phổ biến nhất và rẻ nhất để loại trừ.
●● Độ trễ ổn định tám phút rồi nhảy một bậc?	Chip chạm ngưỡng nhiệt và tụt xung. Con số đúng để báo cáo là con số sau bậc thang đó.
●● Tối ưu lướt xuôi nhanh hơn 30% mà hệ thống chỉ nhanh hơn 7%?	Lướt xuôi chỉ chiếm khoảng một phần tư thời gian. Phần còn lại là giải mã, tiền xử lý, triệt phi cực đại và sao chép giữa CPU với GPU.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ONNX Runtime, TensorRT, OpenVINO	Toàn bộ mạch export → engine → kernel, kể cả optimization profile cho shape động	Hạ tầng; ONNX là định dạng trung gian bền nhất trong nhóm
Triton Inference Server	Batching động kèm trần thời gian chờ — đúng cuộc đánh đổi thông lượng với p99 ở mức batching	Hạ tầng phục vụ; cơ chế batching của nó là hiện thực sạch của bất đẳng thức $B/T_{\text{chờ}}$
vllm, sglang	KV cache và lập lịch yêu cầu cho mô hình sinh, tức đổi bộ nhớ lấy tốc độ ở quy mô nhiều người dùng	Nhóm công cụ đổi nhanh nhất trong danh sách; ý tưởng KV cache thì không đổi
DALI, DeepStream	Đưa giải mã và tiền xử lý lên GPU vì chúng, chứ không phải lượt xuôi, mới chiếm phần lớn thời gian	Đúng kết luận của Hình 1.1, đóng gói thành sản phẩm
DLA trên Jetson Orin	Đẩy một phần đồ thị sang bộ máy khác để giải phóng GPU cho phần còn lại của hệ thống	Chỉ có lái khi toàn bộ nhánh chạy được trên DLA; op không hỗ trợ rơi ngược về GPU và tạo chuyển đổi tốn kém

MINI PROJECT SLAM • Phân rã độ trễ của một pipeline SLAM có thành phần học • 1–2 ngày

Mục tiêu	tìm ra khối nào thật sự ăn thời gian trong hệ của bạn. Rồi đo nó theo cách mà con số còn đúng sau mười phút chạy liên tục, không chỉ trong một trăm lần lặp.
Dữ liệu	hệ SLAM của bạn chạy trên Orin với một chuỗi EuRoC phát lại ở tốc độ thật, có cảm tăng mô tả INT8 từ mini project chương 20 vào frontend.
Các bước	bọc năm khối bằng CUDA event — giải mã và hiệu chỉnh méo, phát hiện đặc trưng, mô tả bằng mạng, ghép cặp cùng RANSAC, và tối ưu — chứ không bấm giờ bằng đồng hồ tường. Bỏ 200 khung hình đầu làm warm-up. Chạy liên tục mười phút ở đúng chế độ nvmodel sẽ triển khai, với các tiến trình thật chạy song song. Ghi lại nhiệt độ và xung nhịp theo thời gian cùng với độ trễ.
Xong khi	bạn chỉ ra được khối chiếm nhiều thời gian nhất, kèm phần trăm của nó. Bạn nói được p99 cao hơn trung bình bao nhiêu phần trăm. Và trên đồ thị độ trễ theo thời gian, bạn chỉ được bậc thang do tụt xung vì nhiệt — hoặc chứng minh bằng đường nhiệt độ rằng nó chưa xuất hiện trong mười phút.
Nối với	D/21 cho hình phân rã thời gian và cuộc đánh đổi thông lượng với p99; B/11 cho lý do bằng thông chung trên Orin làm con số đôi khi có tiến trình khác. Bộ đo dựng ở đây chính là thứ mini project chương 22 đem đi kiểm tính tái lập.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì notebook đo một hệ thống ở trạng thái tốt nhất: đã warm-up hoặc chưa, mát, không có tiến trình nào tranh băng thông, và thường bấm giờ bằng đồng hồ tường quanh một lệnh CUDA bất đồng bộ. Cả bốn sai lệch đều đi cùng một hướng — lạc quan.
- A2. Vào những khối bạn không đo: giải mã, resize và chuẩn hoá, triết phi cực đại, và các bản sao giữa CPU với GPU. Nếu lượt xuôi chỉ chiếm một phần tư thời gian thì 30% của một phần tư đúng bằng 7,5% tổng — số học chứ không phải nghịch lý.
- A3. Vì thông lượng bão hoà trong khi p99 tiếp tục tăng gần tuyến tính theo kích thước batch. Sau một điểm nào đó bạn gần như không mua thêm được thông lượng nhưng vẫn trả đủ giá độ trễ, và nếu tốc độ đến nhỏ hơn $B/T_{\text{chờ}}$ thì bạn còn không lấp nổi batch trong trần chờ.
- A4. Vì phút thứ tám là một trạng thái khác: chip đã chạm ngưỡng nhiệt và tụt xung. Đường độ trễ theo thời gian có một bậc thang, và con số đúng để báo cáo là con số *sau* bậc thang đó.
- A5. Bằng triệu chứng. Lệch do số học thì nhỏ, phân tán đều, cỡ epsilon của fp16 và tích lũy dần theo độ sâu. Lệch do bug thì tập trung ở một tầng và không giảm khi bạn tăng độ chính xác — hoặc lớn và có hệ thống ngay từ tầng đầu, dấu hiệu kinh điển của tiền xử lý lệch.

NÓI LẠI THẬT ĐƠN GIẢN

Một nhà hàng bị chê phục vụ chậm. Chủ quán liền thuê đầu bếp giỏi hơn, nấu nhanh hơn ba mươi phần trăm — và khách vẫn chê. Lý do: nấu chỉ chiếm một phần tư thời gian một món. Phần còn lại là nhặt rau, rửa, thái, xếp đĩa và bưng ra. Muốn nhanh thì phải bấm giờ từng khâu trước đã, chứ không phải cải thiện đúng cái khâu ai cũng nói tới.

Chuyện thứ hai là cách báo cáo. Nói “trung bình hai mươi phút một món” nghe rất ổn, nhưng khách nhớ cái lần họ chờ năm mươi phút. Con số ràng buộc thiết kế là cái lần xui xẻo đó, không phải trung bình.

Chuyện thứ ba là trạng thái của bếp. Bếp lúc mới mở khác hẳn bếp đã chạy tám phút liên tục: nóng lên thì mọi thứ chậm lại. Đo mười lượt lúc bếp còn nguội rồi công bố con số ấy là tự lừa mình.

Chuyện thứ tư là gộp đơn. Nấu một lượt cho tám bàn thì tổng số món phục vụ được nhiều hơn hẳn, nhưng bàn đến trước phải ngồi đợi cho đủ tám. Nếu quán chỉ có một trăm khách một giờ thì chờ cho đủ tám bàn là quá lâu, và bạn đang trả cái giá của gộp đơn mà không nhận được lợi ích của nó.

Cái giá chung của cả chương: mọi con số bạn đo chỉ đúng kèm điều kiện — bếp nóng hay nguội, đông hay vắng, đã chạy thử chưa.

Một câu: Đừng hỏi mô hình chạy nhanh bao nhiêu; hãy hỏi cả hệ thống, ở đúng trạng thái thật, phục vụ được người khách xui xẻo thứ chín mươi chín trong một trăm.

1.12 Kết nối

Chương này nhận đầu vào từ D/20-toi-uu-va-nen-mo-hinh và trả lại cho nó một bài học ngược: mọi phép đo dùng để vẽ đường Pareto ở chương 20 chỉ hợp lệ nếu được đo theo đúng kỷ luật của chương này, nên hai chương thực chất là một vòng lặp chứ không phải một chuỗi. Nó là ứng dụng của B/11-gioi-han-tinh-toan ở phía **suy luận** (*inference*), và nó dựa vào B/08-dich-chuyen-phan-phoi cho toàn bộ lập luận về shadow deployment. Về phía sau, nó là tiền đề của D/22-tai-lap-va-mlops: một khi mô hình đã chạy trong production, câu hỏi đổi từ “nó có nhanh không” sang “sáu tháng nữa tôi có dựng lại được đúng nó không, và tôi có biết nó đang xuống cấp không”. Và nó nối thẳng sang E/25-do-tin-cay-va-van-hanh cho phần hành vi khi mô hình gặp đầu vào ngoài phân phối.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Sau khi export ONNX, đầu ra lệch 10^{-2} — bốn nguyên nhân theo thứ tự cần kiểm là gì, và triệu chứng nào phân biệt “lệch do số học” với “lệch do toán tử bị thay thế”?
2. Vì sao p99 là con số ràng buộc thiết kế còn trung bình thì không, và điều đó đổi cách bạn chọn ngưỡng tin cậy của NMS như thế nào?
3. Với ngân sách p99 là 40 ms và lượt xuôi batch 8 tốn 20 ms, tốc độ đến tối thiểu để batching động có lãi là bao nhiêu, và dưới ngưỡng đó cấu hình đúng là gì?
4. Vì sao một benchmark một trăm lần lặp trên Jetson nói dối, và phép đo tối thiểu để không bị nó lừa gồm những điều kiện nào?
5. Vì sao engine TensorRT phải dựng lại sau khi nâng JetPack, và vì sao “dựng lại” là chưa đủ?
6. Shadow deployment bắt được loại lỗi nào mà tập test không bắt được, và loại lỗi nào nó vẫn không bắt được?

Nguồn

- [1] D. Sculley, G. Holt, D. Golovin, et al., “Hidden Technical Debt in Machine Learning Systems,” *NeurIPS*, 2015.
- [2] N. Bodla, B. Singh, R. Chellappa, et al., “Soft-NMS: Improving Object Detection with One Line of Code,” in *ICCV*, 2017.